

# Reinforcement Learning for Algorithmic Trading

WiDS Final Project Report

Sahil M Deo, First Year Undergraduate (CSE), IIT Bombay

Winter in Data Science (WiDS) Program

## Abstract

This report documents my learning journey and project work carried out as part of the Winter in Data Science (WiDS) program. Coming from a first-year undergraduate background with limited prior exposure to machine learning and finance, this program served as an intensive introduction to both reinforcement learning theory and its practical applications.

The project evolved over four weeks. The initial weeks focused on building programming skills and understanding fundamental reinforcement learning concepts such as bandit problems and Markov Decision Processes. The final week culminated in implementing a Q-Learning based stock trading agent and evaluating its performance against a classical baseline strategy.

The objective of this report is to describe the theoretical background, implementation details, experimental methodology, and key lessons learned during the project. The content is based on the accompanying GitHub repository and the final project notebook.

## 1. Introduction

Reinforcement Learning (RL) is a subfield of machine learning concerned with training agents to make sequential decisions through interaction with an environment. Unlike supervised learning, RL does not rely on labeled input-output pairs. Instead, the agent learns by trial and error, guided by a reward signal.

Financial markets provide an interesting and challenging environment for reinforcement learning. Price movements are noisy, partially unpredictable, and influenced by numerous hidden factors. Despite this complexity, simplified market models are useful educational tools for understanding how learning algorithms behave in dynamic systems.

As a first-year student, my motivation for selecting this project was to gain hands-on exposure to both Python-based data analysis and reinforcement learning, areas that are increasingly relevant in computer science research and industry. The WiDS program provided a structured pathway to build these skills from the ground up.

This report is organized as follows. Sections 2 to 4 describe the tools used and early work on bandit algorithms and MDPs. Sections 5 and 6 discuss financial data analysis and Q-Learning theory. Sections 7 and 8 present the trading environment, training procedure, and evaluation results. Finally, limitations and future directions are discussed.

## 2. Tools and Technical Stack

The project was implemented entirely in Python due to its simplicity, readability, and rich ecosystem of scientific libraries. Prior to this program, my experience with Python was limited, and learning to use it effectively was a major component of the first week.

NumPy was used for efficient numerical computations, especially while implementing bandit algorithms and storing Q-tables. Its array-based operations enabled concise representation of value functions and reward statistics.

Pandas was used extensively for handling stock market datasets. It provides convenient abstractions such as DataFrames and time-indexed series, which made it easy to clean data, compute indicators, and iterate over price sequences.

Matplotlib was used to visualize reward curves, portfolio value over time, and comparative performance between different strategies. Visualization played an important role in debugging and interpreting the learning behavior of algorithms.

GitHub was used to maintain version control and document progress. Learning to use git for committing code, writing meaningful README files, and organizing notebooks was itself an important professional skill gained through this project.

### 3. Multi-Armed Bandit Algorithms

The multi-armed bandit problem is one of the simplest models of decision making under uncertainty. An agent repeatedly selects one of several arms, each associated with an unknown reward distribution. The goal is to maximize cumulative reward over time.

I implemented three commonly studied strategies: greedy, epsilon-greedy, and Upper Confidence Bound (UCB). The greedy strategy always selects the arm with the highest estimated reward, which often leads to premature convergence to suboptimal arms.

The epsilon-greedy strategy introduces controlled randomness by choosing a random arm with probability epsilon and exploiting the best-known arm otherwise. This balances exploration and exploitation more effectively.

The UCB strategy uses statistical confidence bounds to guide exploration, selecting arms with high uncertainty even if their estimated reward is lower. This theoretically grounded approach performed consistently well in simulations.

These experiments provided practical intuition about why exploration is essential and how naive exploitation can result in long-term losses.

## 4. Markov Decision Processes

Markov Decision Processes (MDPs) generalize bandit problems by introducing states and state transitions. An MDP is defined by a set of states, actions, transition probabilities, rewards, and a discount factor.

I modeled several classical environments such as Bandit Walk, Slippery Walk, and Frozen Lake as MDPs. Each environment involved stochastic transitions and delayed rewards, making planning more complex than in bandit settings.

To avoid manually specifying transition tables for every environment, I wrote Python utilities to automatically generate transition matrices based on grid structure and movement rules. This significantly reduced boilerplate code and improved scalability.

Understanding MDPs was crucial for later modeling the trading problem, since stock trading naturally involves sequential decisions influenced by past actions and market evolution.

## 5. Financial Data and Baseline Trading Strategy

Before applying reinforcement learning to trading, it was important to understand basic financial terminology and market data formats. I studied common price attributes such as Open, High, Low, Close, and Adjusted Close.

Using historical stock price data, I implemented a simple moving average crossover strategy. The idea is to buy when a short-term average crosses above a long-term average and sell when the opposite occurs.

This baseline strategy helped in understanding how algorithmic trading rules can be expressed programmatically and how profits and losses accumulate over time.

It also provided a useful reference point when evaluating the performance of the reinforcement learning agent in the final week.

## 6. Q-Learning Theory

Q-Learning is a model-free reinforcement learning algorithm that aims to learn the optimal state-action value function  $Q(s,a)$ . This function represents the expected cumulative reward obtained by taking action  $a$  in state  $s$  and following the optimal policy thereafter.

The update rule is derived from the Bellman optimality equation and incrementally adjusts Q-values using observed rewards and estimates of future returns.

A key advantage of Q-Learning is that it does not require knowledge of transition probabilities, making it suitable for environments where the dynamics are unknown or complex.

However, tabular Q-Learning scales poorly with large or continuous state spaces, which is an important limitation when dealing with realistic financial markets.

## 7. Trading Environment Design

The trading environment was implemented as a custom Python class that simulates interaction with a stock market using historical price data.

The action space consisted of three actions: buy one unit of stock, sell one unit of stock, or hold the current position.

The state representation was derived from the current timestep and price information. Although simple, this formulation allowed experimentation with reinforcement learning concepts without excessive complexity.

The agent started with an initial capital of 100,000 units. At each step, the reward was defined as the total portfolio value, computed as the sum of remaining cash and the market value of held shares.

An episode terminated when the price series ended, at which point the final portfolio value represented the outcome of the policy.



## 8. Training Procedure and Evaluation

The agent was trained over multiple episodes using an epsilon-greedy exploration strategy with epsilon set to 0.1. This ensured continued exploration while gradually favoring profitable actions.

At each timestep, the agent selected an action, executed it in the environment, observed the resulting reward and next state, and updated the Q-table accordingly.

After training, the learned policy was evaluated by running the agent on the same historical data without exploration and recording the portfolio value trajectory.

A buy-and-hold strategy was implemented as a baseline for comparison. While the reinforcement learning agent demonstrated adaptive behavior, its performance varied depending on market conditions and hyperparameter choices.

These experiments highlighted both the potential and the fragility of simple reinforcement learning approaches in financial domains.

## 9. Discussion and Limitations

This project successfully demonstrated the end-to-end pipeline of building a reinforcement learning system, from environment modeling to policy evaluation.

However, several limitations remain. The state representation was simplistic and did not capture technical indicators or long-term trends. Transaction costs and market impact were also ignored.

Furthermore, tabular Q-Learning is not suitable for high-dimensional or continuous state spaces, motivating the use of neural networks and deep reinforcement learning methods in future work.

Despite these limitations, the project served as a valuable educational exercise and strengthened my understanding of both reinforcement learning theory and Python-based experimentation.

## 10. Conclusion and Future Work

The WiDS program provided a structured and practical introduction to reinforcement learning, starting from simple bandit problems and culminating in a complete trading agent.

As a first-year undergraduate student, this experience significantly improved my confidence in implementing complex algorithms and reading research-oriented material.

In future work, this project could be extended using deep Q-networks, richer state representations, multiple assets, and realistic market constraints.

Overall, the project bridged the gap between theoretical concepts and real-world inspired applications, making it a valuable learning milestone in my academic journey.